ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH

THỰC TẬP ĐỒ ÁN ĐA NGÀNH
HƯỚNG TRÍ TUỆ NHÂN TẠO

Báo cáo 2: Khảo sát cách dùng LLM để tối ưu hóa hiệu năng các ứng dụng AI hiện nay, phân tích các ưu điểm – nhược điểm, các vấn đề tồn đọng và phương pháp giải quyết

Giảng viên phụ trách: Ths. Hoàng Lê Hải Thanh

Thực hiện: Nguyễn Ánh Dương – 2310596

Mục lục

I. Giới thiệu 2
II. Các phương pháp tiếp cận 3
1. SwizzlePerf: Hardware-Aware LLMs cho tối ưu hóa khi chạy GPU kernel [1] 3
2. KernelBand: Steering LLM-based Kernel Optimization via Hardware-Aware Multi-Armed Bandits [2] 5
III. Kết quả thực nghiệm 9
1. Kết quả của SwizzlePerf [1] 9
2. Kết quả KernelBand [2] 11
IV. Đánh giá các phương pháp 13
1. KernelBench Framework [5] 13
2. MEP Framework [3] 15
3. ASTRA Framework [4] 16
4. SwizzlePerf framework [1] 18
5. KernelBand framework [2] 19
V. So sánh giữa nhóm MEP – Astra và nhóm SwizzlePerf – KernelBand 20
1. Nhóm Astra và MEP 20
2. Nhóm KernelBand và SwizzlePerf 20
TÀI LIỆU THAM KHẢO 21

I. Giới thiệu

Trong lĩnh vực HPC và AI, các GPU là quan trọng đến hiệu suất của toàn bộ hệ thống. Ngoài việc tối ưu hóa mã nguồn GPU thì việc tối ưu các khịa cạnh khác như tối ưu sử dụng tài nguyên khi chạy chương trình và tối ưu hiệu quả phân bổ tài nguyên GPU trong hệ thống lớn cũng cực kì quan trọng. Báo cáo này tập trung khảo sát các phương pháp **tối ưu phần cứng và tối ưu phân bổ tài nguyên GPU** sử dụng Mô hình ngôn ngữ lớn (LLM). Bài khảo sát này là tiếp nói của bài khảo sát 1. Bài khảo sát 2 này khảo sát thêm 2 bài báo dựa trên 2 bài báo đều được công bố trên ***Arxiv*** của ***Conrnel University,*** ngoài ra bài khảo sát 2 này cũng review lại 3 bài báo và 2 phương pháp đề cập trong bài khảo sát 1 cùng với 2 phương pháp mới để phân tích ưu điểm – nhược điểm và các vấn đề liên quan, cũng như tìm kiếm các hướng giải quyết nếu có. Hai bài báo được đề cập trong bài khảo sát này có điểm chung là tập trung chi tiết với metric là tập trung vào một hoặc nhiều đặc điểm của phần cứng, bám sát phần cứng để tối ưu code, trong khi ***Astra*** và ***MEP*** ở bài trước không có cơ chế Hardware-aware như thế này.

- SwizzlePerf: Hardware-Aware LLMs cho tối ưu hóa khi chạy GPU kernel [1]
- KernelBand: Steering LLM-based Kernel Optimization via Hardware-Aware Multi-Armed Bandits [2]

Lưu ý: Tất cả các tiêu đề có đánh chỉ mục tài liệu tham khảo, thì tất cả các hình ảnh, công thức và diễn giải trong mục đó đều được chụp, trình bày lại từ chính tài liệu tham khảo.

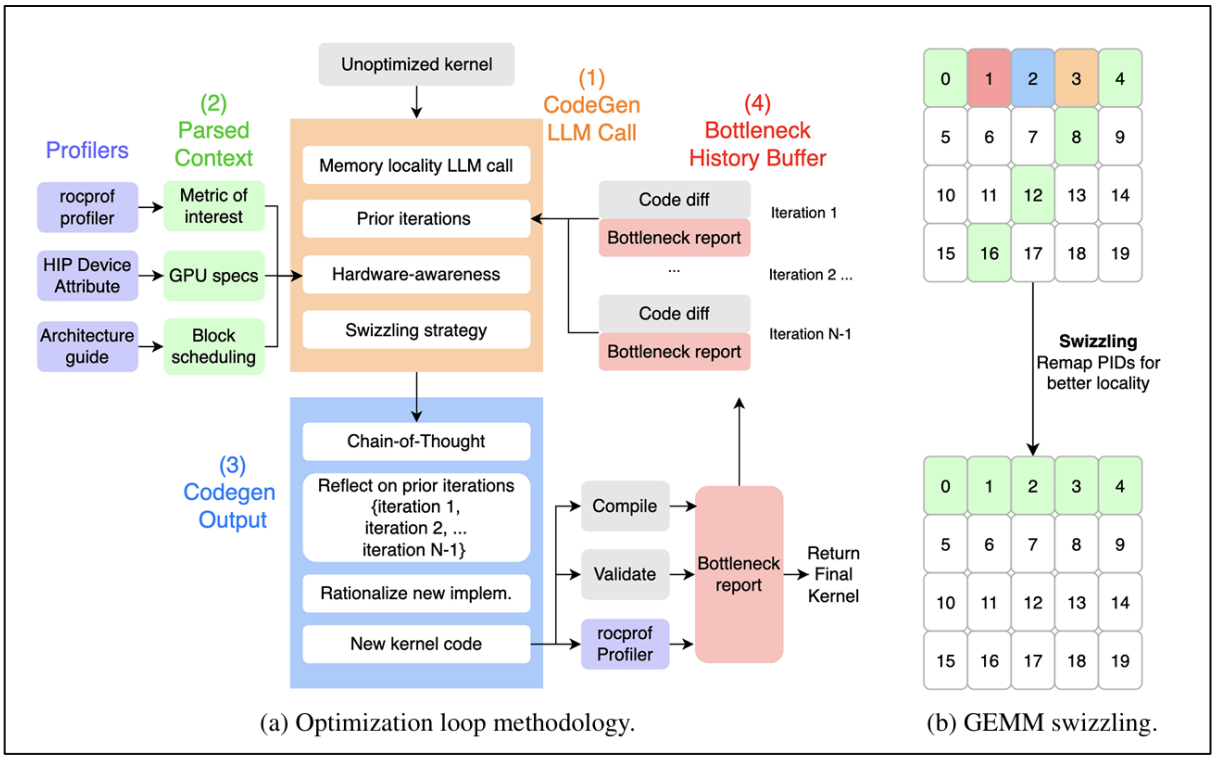
II. Các phương pháp tiếp cận

1. SwizzlePerf: Hardware-Aware LLMs cho tối ưu hóa khi chạy GPU kernel [1]

Các phương pháp tối ưu trước đây thường dựa nhiều vào kết quả thời gian chạy. Tuy nhiên, hầu hết các phương pháp không có một metric đánh giá cụ thể là nên tối ưu kernel dựa trên khía cạnh gì của phần cứng, xem xét toàn bộ khía cạnh khiến cho dữ liệu thu được của runtime dễ bị nhiễu.

SwizzlePerf thực chất là cung cấp cho LLM nhận thức sâu sắc về phần cứng. Thay vì tối ưu runtime chung chung, hệ thống nhắm thẳng vào một ***chỉ số nút thắt cổ chai "bottleneck" metric*** cụ thể. Trong nghiên cứu này, chỉ số đó là ***tỷ lệ hit-miss nhớ đệm L2.***

Phương pháp này tập trung vào kỹ thuật **Swizzling,** bằng việc tính toán và ánh xạ lại ID của các khối xử lý workgroup PIDs nhằm gom các tính toán có chung dữ liệu vào cùng một phân vùng phần cứng XCD trên các kiến trúc GPU phân mảnh như AMD MI300X ***(thay đổi thứ tự ánh xạ của các block tính toán vào các vi xử lý)***



(a) Optimization loop methodology. (b) GEMM swizzling.

Hình 1. Pipeline của SwizzlePerf

Quy trình của SwizzlePerf là vòng lặp bao gồm 4 thành phần chính:

- **Trích xuất ngữ cảnh phần cứng (2):** Hệ thống thu thập thông số trực tiếp từ công cụ như ***rocprof*** của AMD, thông số GPU gồm *kích thước cache, số lượng XCD,…* và *block-scheduling policy*.
- **Prompt thiết kế đặc thù (1):** LLM input prompt chứa mô tả về thuật toán, thông số phần cứng cụ thể và lịch sử các lần tối ưu thất bại trước đó và thông in về phần cứng rồi đưa ra chiến lược swizzling

Input prompt

The original code is {...} with bottleneck {...}
The memory analysis is {...}
History of previous optimization attempts (do not repeat an implementation): {
  iteration: 1, applied diff: {...}, bottleneck report: {...},
  iteration: N-1, applied diff: {...}, bottleneck report: {...}
}
On the AMD MI300X, there are {A} XCDs, each has {B} CUs and a {C} MB L2 cache
Blocks are scheduled {Round-robin to XCDs}
Your swizzling goal is to {...}, pay special attention to {...}, code should be structured as {...}

Hình 2. Input Prompt ở bước CodeGen LLM call

- **Sinh mã suy luận (3):** Sử dụng ***framework DSPy***, LLM bị bắt buộc phải sử dụng kỹ thuật ***Chain-of-Thought*** để giải thích tại sao đoạn code cũ thất bại trước khi sinh ra công thức swizzling mới. Sau đó chạy lại code mới, đánh giá.

Output Signature

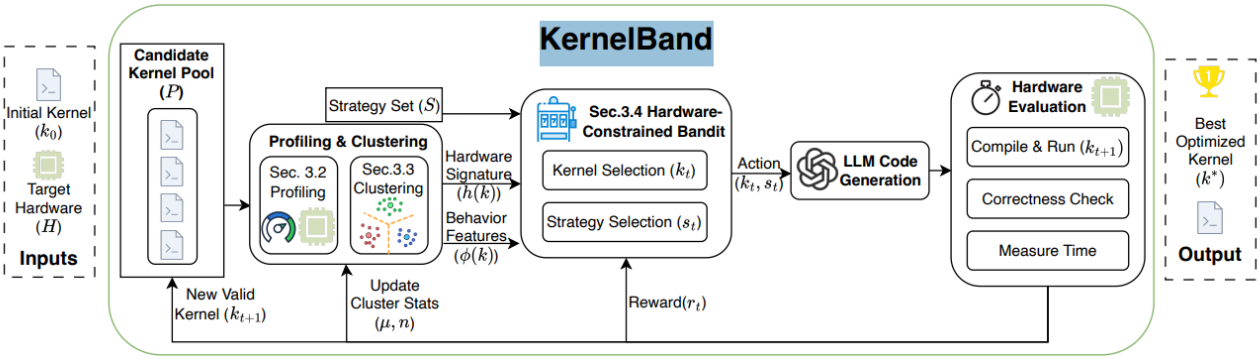
Chain-of-Thought Reasoning
JSON dict of why old implementations were suboptimal: {
  iteration 1: This implementation only marginally increased L2 hit rate because {...},
  iteration N-1: This iteration didn't change L2 hit rate because {...}
}
My new swizzling approach will remap blocks by {...}
My new swizzling approach will be better than any prior implementation because {...}
Final code

Hình 3. Quy trình ở bước CodeGen Ouput

- **Bộ đệm lịch sử (4):** Mỗi khi bước (3) hoàn thành một vòng chạy, nó sẽ gói tổng hợp thay đổi của code (Code diff) và Báo cáo hiệu năng mới (Bottleneck report) đẩy vào bộ đệm là Bottleneck History Buffer. Nếu code chạy sai, hoặc L2 hit rate không tăng, thông tin này được lưu lại. Ở vòng lặp tiếp theo (quay lại bước (1) ở prior iterations), nó lặp qua các report trước đó (Hình 2), điều này tạo thành một vòng lặp khép kín, giúp LLM liên tục nhận được feedback thực tế từ phần cứng và lịch sử code và bottleneck sửa sai và hội tụ về đáp án tối ưu.

2. KernelBand: Steering LLM-based Kernel Optimization via Hardware-Aware Multi-Armed Bandits [2]

KernelBand tập trung tối ưu hóa giống dựa trên bài toán **Multi-Armed Bandit trong xác suất thống kê**. Hệ thống phải chọn xem nên áp dụng chiến lược tối ưu nào lên đoạn code nào để tối đa hóa hiệu năng.



Hình 4. Overview về pipeline của KernelBand

Hệ thống hoạt động qua 3 cơ chế chính:

- **Trace-Driven Clustering:** Thay vì quản lý từng kernel riêng thì hệ thống trích xuất 5 đặc trưng thời gian chạy (như số thanh ghi, shared memory, occupancy) và dùng thuật toán K-Means để gom nhóm các kernel có chung bottleneck chung thành các cụm.
- **Hardware-Aware Pruning:** Dùng công cụ NVIDIA Nsight Compute để đo lường. Nếu một tài nguyên như băng thông bộ nhớ đã đạt mức bão hòa thì hệ thống sẽ loại bỏ các chiến lược cố gắng tối ưu tài nguyên đó để tránh lãng phí.

- **Upper Confidence Bound:** Thuật toán UCB sẽ chọn nên dùng chiến lược đã từng thành công hay thử chiến lược mới. Sau khi chọn được chiến lược, LLM mới được gọi để viết code thực thi chiến lược đó.

  Hình trên là chi tiết về thuật toán của KernelBand và dưới đây là sẽ đi sâu vào giải thích chi tiết thuật toán.

Algorithm 1 Workflow of KERNELBAND

Require: Kernel k_0, Strategies $\mathcal{S}$, Budget T, Period $\tau = 10$, Clusters K

$\mathcal{P} \leftarrow \{k_0\}, \mathcal{C} \leftarrow \{\mathcal{P}\}$ ▷ *Initial single cluster*
Initialize $N_{i,s} \leftarrow 1, \hat{\mu}_{i,s} \leftarrow 0.5$ for all i, s
Initialize Masks $M_{i,s} \leftarrow 1$ ▷ *No pruning initially*
for $t = 1$ **to** T **do**
  Compute $\phi(k)$ for all $k \in \mathcal{P}$
  ▷ ***Periodic clustering & profiling***
  if $t \bmod \tau = 0$ **and** $|\mathcal{P}| \geq 2K$ **then**
    $\mathcal{C} \leftarrow \text{KMeans}(\{\phi(k)\}, K)$
    Update centroids $k_c^{(i)}$ and profile $h_c^{(i)} \leftarrow \text{NCU}(k_c^{(i)})$
  end if
  ▷ ***Hardware-constrained selection***
  if centroids are profiled **then**
    $M_{i,s} \leftarrow \mathbb{I}[h_c^{(i)}[\text{Target}(s)] < \theta_{\text{sat}}]$
  end if
  Select (I_t, S_t) via Eq. (6) (Masked UCB)
  Sample $k_t \in C_{I_t}$ with $P(k) \propto \exp(V_{\text{hw}}(k, S_t))$
  ▷ ***Generation & update***
  $k'_t \leftarrow \text{LLM}(k_t, S_t)$
  if $\text{Verify}(k'_t)$ **then**
    $r_t \leftarrow \max(0, (\mathcal{T}(k_t) - \mathcal{T}(k'_t))/\mathcal{T}(k_t))$
    $\mathcal{P} \leftarrow \mathcal{P} \cup \{k'_t\}$
    $N_{I_t,S_t} \leftarrow N_{I_t,S_t} + 1$
    $\hat{\mu}_{I_t,S_t} \leftarrow \hat{\mu}_{I_t,S_t} + \frac{r_t - \hat{\mu}_{I_t,S_t}}{N_{I_t,S_t}}$
  end if
end for
return $\arg\min_{k \in \mathcal{P}} \mathcal{T}(k)$

Hình 5. Chi tiết thuật toán MAB của KernelBand

❖ *Bước 1: Rút trích đặc trưng thời gian chạy*

Để thuật toán MAB có thể học thì nó cần biết mỗi đoạn code kernel đang hoạt động ở trạng thái như thế nào. Hệ thống thu thập 2 loại thông tin cho mỗi kernel ***k***:

1. Vector đặc trưng:

$$\emptyset(\boldsymbol{k}) = \left[\widetilde{\boldsymbol{T}}(\boldsymbol{k}), \boldsymbol{n}_{reg}, \boldsymbol{n}_{smem}, \boldsymbol{d}_{block}, \boldsymbol{\eta}_{occ}\right]$$

   Đây là một vector 5 chiều đại diện cho thông tin của kernel khi chạy, bao gồm: Thời gian chạy đã chuẩn hóa $\widetilde{\boldsymbol{T}}(\boldsymbol{k})$, số thanh ghi mỗi luồng $\boldsymbol{n}_{reg}$, bộ nhớ dùng chung mỗi block $\boldsymbol{n}_{smem}$, kích thước block $\boldsymbol{d}_{block}$, và tỷ lệ lấp đầy trên lý thuyết $\boldsymbol{\eta}_{occ}$.

2. Thông tin về phần cứng ***h(k)***:
   Sử dụng công cụ ***NVIDIA Nsight Compute*** để đo lường tỷ lệ băng thông đã sử dụng của bộ nhớ, Cache L2, và nhân tính toán. Nó giúp hệ thống biết chính xác kernel đang bị nghẽn ở đâu.

❖ ***Bước 2: Dynamic Clustering***

Nếu hệ thống lưu lịch sử cho hàng trăm kernel sinh ra, thuật toán Bandit sẽ bị tràn dữ liệu và thiếu hiệu quả tối ưu. Do đó***, hệ thống gom nhóm chúng lại***: Cứ sau mỗi chu kỳ τ, hệ thống dùng thuật toán ***K-Means*** để phân cụm tất cả các kernel hiện có thành ***K*** nhóm dựa trên vector $\emptyset(\boldsymbol{k})$ ở trên. Khi cập nhật thông tin, thay vì đo Nsight Compute cho toàn bộ kernel trong cụm rất tốn thời gian, hệ thống chỉ đo của kernel ở vị trí trung tâm centroid $k_c^{(i)}$ của mỗi cụm.

❖ ***Bước 3: Hardware Constraint Selection***

Tại mỗi vòng lặp ***t***, thuật toán sẽ cần chọn cụm kernel I_t và áp dụng chiến lược S_t để giao cho LLM viết code. Nó sẽ thực hiện qua 2 công đoạn:

1. Loại bỏ bão hòa:

$$\boldsymbol{M}_{i,s} = \mathbb{I}\left[\boldsymbol{h}(\boldsymbol{k}_c^{(i)})[\boldsymbol{Target}(\boldsymbol{s})] < \boldsymbol{\theta}_{sat}\right]$$

   Hệ thống tạo ra một mặt nạ $\boldsymbol{M}_{i,s}$ (giá trị 0 hoặc 1). Nếu tài nguyên phần cứng mà chiến lược ***s*** nhắm tới đã đạt ngưỡng bão hòa $\boldsymbol{\theta}_{sat}$ (ví dụ là 75%), Mask sẽ bằng 0. Ví dụ: Nếu băng thông bộ nhớ đã chạy hết công suất 80%, hệ thống không cho LLM áp dụng chiến lược

Vector hóa nữa (vì Vector hóa chỉ làm tăng thêm áp lực lên băng thông, trong khi nó không thể được tối ưu nữa).

2. Chọn cặp kernel và chiến lược để LLM sinh code:

$$(I_t, S_t) = arg\ max_{i \in [K], s \in S} (\hat{\mu}_{i,s} + c\sqrt{\frac{ln(t)}{N_{i,s}}})$$

Dùng thuật toán Upper Confidence Bound (UCB), cân bằng giữa 2 yếu tố:

$\hat{\mu}_{i,s}$**:** Điểm phần thưởng trung bình mà chiến lược s đã đạt được trên cụm i trong quá khứ.

$c\sqrt{\frac{ln(t)}{N_{i,s}}}$ **:** Phép tính này ưu tiên những chiến lược chưa được thử nhiều ($N_{i,s}$ nhỏ). Càng ít được thử, hệ số này càng cao, khiến thuật toán sẽ phải thử nghiệm hướng đi mới thay vì áp dụng chiến lượt cũ nhiều.

Sau khi chọn được Cụm (I_t, S_t), hệ thống sẽ bốc ngẫu nhiên một kernel trong cụm đó mà ưu tiên kernel còn nhiều không gian trống để tối ưu phần cứng. Gửi lệnh này cho LLM viết code. Nếu code chạy đúng và nhanh hơn code cũ, hệ thống tính toán hệ số thưởng r_t bằng tỷ lệ thời gian được rút ngắn:

$$r_t = max(0, \frac{T(k_t) - T(k'_t)}{T(k_t)})$$

Sau đó cập nhật phần thưởng này vào $\hat{\mu}_{i,s}$ để thuật toán MAB học cho vòng sau.

III. Kết quả thực nghiệm

1. Kết quả của SwizzlePerf [1]

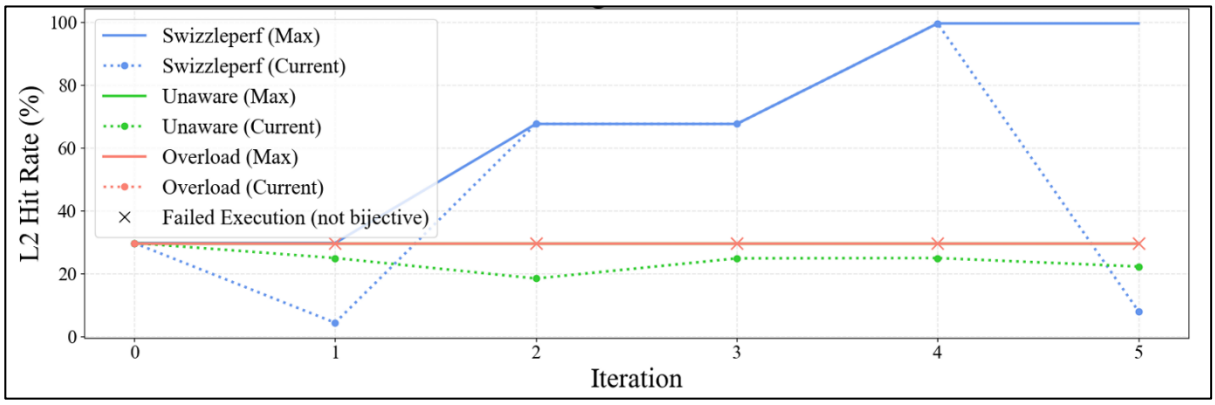
a. *Kết quả tổng quát:*

Hệ thống SwizzlePerf được đánh giá trên 10 kernel phổ biến, chia làm 2 nhóm: 6 kernel Machine Learning như GEMM, LayerNorm, Softmax và 4 kernel Khoa học như Black-Scholes, FDTD 2D.

```
pid_m = tl.program_id(0)
pid_n = tl.program_id(1)

num_blocks_m = (M + BLOCK_SIZE_M - 1) // BLOCK_SIZE_M
num_blocks_n = (N + BLOCK_SIZE_N - 1) // BLOCK_SIZE_N
total_blocks = num_blocks_m * num_blocks_n

block_id = pid_m * num_blocks_n + pid_n
xcd_id = (block_id // num_blocks_m) % 8
round_id = block_id // (8 * num_blocks_m)
new_block_id = round_id * 8 + xcd_id

pid_m = new_block_id // num_blocks_n
pid_n = new_block_id % num_blocks_n
```

Hình 6. Code Kernel mà SwizzlePerf sinh ra cho bài toán stencil 2D

Kết quả báo cáo cho thấy SwizzlePerf đạt mức speedup trung bình là **1.29x**, trong đó cao nhất lên tới **2.06x** trên kernel Transpose. Tỷ lệ trúng bộ nhớ đệm L2 được cải thiện trung bình **23.9%** và cao nhất lên tới **70%**. Bài báo cũng đồng thời khảo sát hai phương pháp là "Hardware-unaware" (Không có thông tin phần cứng) và "Hardware-overload" (Cung cấp rất nhiều thông tin vào LLM) thì đều gần như không cải thiện được L2 hit rate và không mang lại bất kỳ sự tăng tốc nào.

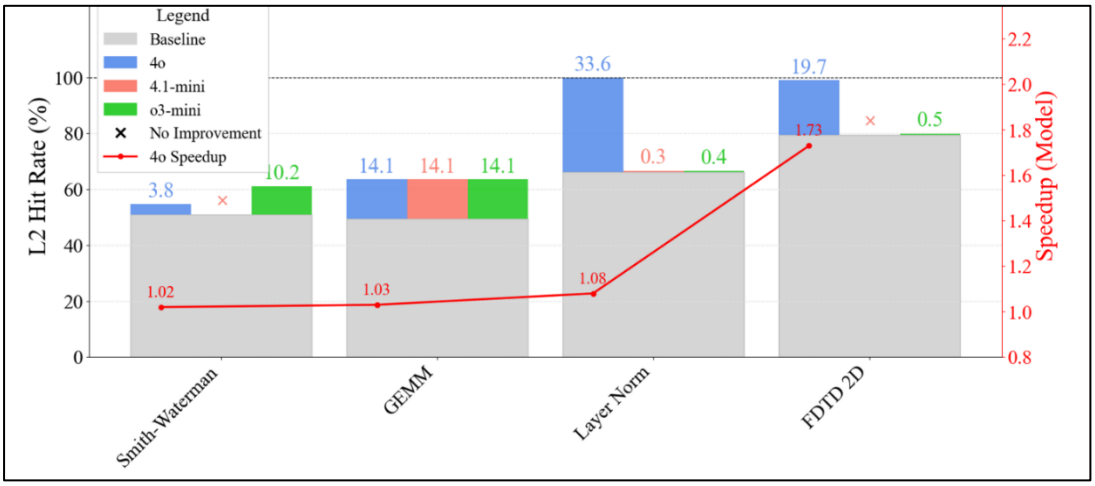


Hình 7. Biểu đồ L2 Hit Rate của kernel Stencil 2D kernel mỗi vòng lặp so với 3 baseline là Hardware-aware (xanh dương), Hardware-unaware (xanh lá cây), Hardware-overload (đỏ)

Một ví dụ là kết quả chạy kernel Stencil 2D cho thấy trong 5 vòng lặp, Hardware-aware có L2 Hit Rate tốt nhất là hơn 100% (nét liền) tại vòng lặp 4, còn đến vòng 5 thì chỉ còn 20% (nét đứt). Đồ thị cũng cho thấy, phương pháp Hardware-aware là vượt trội nhất, trong khi Hardware-overload cho ra kết quả Failed Execution và Hardware-unaware do không có kiến thức về phần cứng nên L2 Hit Rate là cực thấp.

b. *Khả năng Hardware-aware của các LLM khác nhau:*

Tác giả đã chạy thử với OpenAI ***GPT-4o, GPT-4o-mini, và o3-mini***



Hình 8. Biểu đồ L2 Hit Rate - Speedup khảo sát 3 LLM với các 4 kernel khác nhau

Với các kernel đơn giản như GEMM thì cả 3 mô hình đều tìm ra được công thức Swizzling tối ưu. Trên các kernel phức tạp hơn như LayerNorm và FDTD, chỉ có mô hình GPT-4o là tìm ra cách giải. Mô hình o3-mini chuyên về suy luận toán học và code lại kém hiệu quả hơn GPT-4o thông thường trong bài LayerNorm. Tuy nhiên, ở bài toán khó là Smith-Waterman, o3-mini lại tạo ra Swizzling tối ưu hơn phiên bản của 4o.

Việc các LLM được train thuần túy để suy luận logic không phải lúc nào cũng mang lại hiệu quả tốt nhất khi bài toán yêu cầu phải có ngữ cảnh phần cứng rõ ràng. Việc lựa chọn đúng LLM sẽ ***phụ thuộc nhiều vào từng loại kernel và kiến trúc cụ thể***.

2. Kết quả KernelBand [2]

a. Hiệu năng trên các GPU khác nhau:

Platform	Method	L1–2 C	L1–2 F	L1–2 G	L3 C	L3 F	L3 G	L4–5 C	L4–5 F	L4–5 G	All C	All F	All G
RTX 4090	BoN	63.6	9.1	0.86	12.1	6.1	0.89	28.6	14.3	1.09	31.1	10.0	0.96
	GEAK	68.2	36.4	1.62	21.2	18.2	1.45	80.0	40.0	1.34	55.6	31.1	1.44
	KERNELBAND	**81.8**	**50.0**	**2.14**	**60.6**	**30.3**	**1.82**	**91.4**	**51.4**	**1.47**	**77.8**	**43.3**	**1.74**
H20	BoN	65.5	34.5	1.41	36.9	23.1	**1.35**	7.1	1.4	0.63	29.3	15.9	0.99
	GEAK	68.9	34.5	1.29	60.0	29.2	1.16	31.4	14.3	0.90	49.4	23.8	1.06
	KERNELBAND	**93.1**	**51.7**	**1.70**	**86.2**	**63.1**	**1.35**	**67.1**	**54.3**	**1.46**	**79.3**	**57.3**	**1.45**
A100	BoN	73.9	34.8	1.43	38.7	24.2	1.23	14.8	6.8	0.76	31.2	16.8	0.98
	GEAK	65.2	43.5	1.09	54.8	45.2	1.18	38.6	33.0	1.54	48.0	38.7	1.34
	KERNELBAND	**95.7**	**82.6**	**2.23**	**91.9**	**77.4**	**2.05**	**67.1**	**42.1**	**1.75**	**79.8**	**60.1**	**1.91**

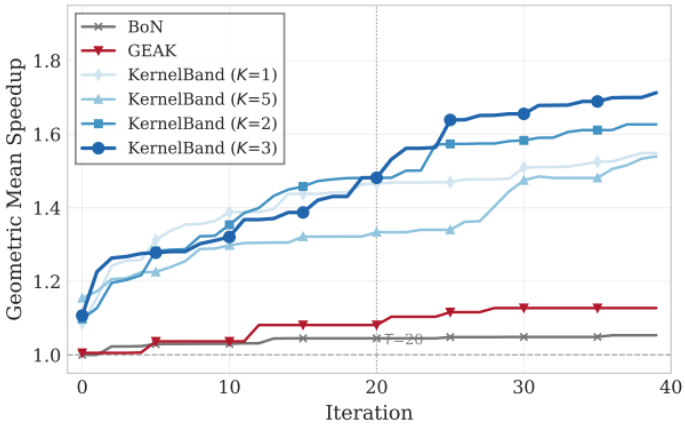
Hình 9. Bảng thống kê so sánh các phương pháp trên các GPU khác nhau. Giải thích các chú thích: C tỷ lệ đúng, F tỷ lệ Fast@1 là tỷ lệ code sinh ra nhanh hơn code gốc, G là Geometric mean speedup hay speedup trung bình nhân, còn L1-5 là mức độ khó của kernel sinh ra

- ***Trên GPU A100***: KernelBand đạt tốc độ tăng tốc 1.91x với tỷ lệ code đúng là 79.8%, trong khi là GEAK chỉ đạt 1.34x và 48.0% đúng.

- ***Tỷ lệ Fast@1***: thuật toán GEAK chỉ có 38.7% cơ hội sinh ra được một đoạn code làm code chạy nhanh hơn trên A100, KernelBand tỷ lệ 60.1%.

- ***Kernel khó L4-5***: Ở các kernel phức tạp L 4-5, BoN chỉ 14.8% đúng, GEAK 38.6% đúng, 1.54x, KernelBand ổn định 67.1% đúng, 1.75x. Thuật toán Bandit hoạt động tốt khi không gian mà code kernel chạy trở nên phức tạp.

b. *Đánh giá hiệu quả thuật toán của KernelBand:*



Hình 10. Thể hiên tốc độ tăng tốc của trung bình nhân nếu cho phép hệ thống chạy tối ưu hóa nhiều

Cả đồ thị của BoN và GEAK đều đi ngang ở khoảng vòng lặp thứ, tức là chạy thêm chỉ tốn chi phí API vô ích vì LLM không thể tìm ra chiến lược tối ưu nữa. Đồ thị của KernelBand đi lên và đạt 1.71x ở vòng lặp 39. Tức là thuật toán Bandit hoạt động tốt nhờ lưu các chiến lược mới để áp dụng dần dần thay vì thử hết một lúc.

IV. Đánh giá các phương pháp

Phần này bổ sung thêm phần xem xét ưu điểm và nhược điểm của 3 bài báo trước liên quan đến MEP framework, Astra framework và KernelBench.

1. KernelBench Framework [5]

Lưu ý: KernelBench Framework là một khảo sát đo lường độ hiểu quả trong việc sinh code kernel giữa các mô hình LLM với nhau.

a. *Các issue mà bài báo đề cập:*

Tỷ lệ lỗi thực thi và sai logic cực cao: Các LLM gặp khó khăn lớn trong việc sinh ra mã nguồn chạy được. Các lỗi bao gồm lỗi biên dịch NVCC, lỗi tràn bộ nhớ CUDA, và sai lệch kết quả toán học Functional Correctness.

Hiệu năng kém hơn thư viện gốc: Dưới 20% số đoạn code do LLM sinh ra có thể chạy nhanh hơn tốc độ mặc định của PyTorch Eager.

Hardware Blindness: Code do LLM sinh ra thông tin về phần cứng; một đoạn kernel chạy nhanh trên NVIDIA L40S có thể chạy rất chậm hoặc sinh lỗi trên NVIDIA A10G. Các mô hình cũng hiếm khi tự động áp dụng các kỹ thuật tối ưu phần cứng như chia khối tiling hay gộp toán tử fusion.

b. *Các hướng giải quyết các issue mà bài báo đề xuất:*

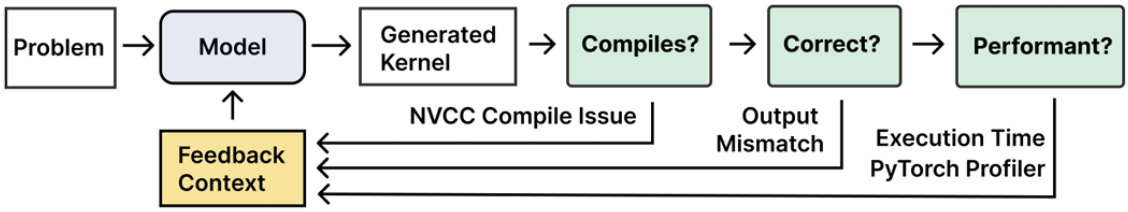
Để nâng cao chất lượng code sinh ra, KernelBench đã đề xuất và thử nghiệm 3 kỹ thuật chính :

Kỹ thuật 1: Repeated Sampling (Đã trình bày trong bài khảo sát trước)

Thay vì chỉ sinh 1 lần, hệ thống cho phép LLM sinh ra hàng chục đến hàng trăm đoạn code (lên tới k=100) với tham số temperature cao để tăng tính đa dạng. Điều này Giúp LLM khám phá không gian giải pháp tốt hơn, tăng vọt tỷ lệ bắt gặp code tối ưu hơn. Kết quả: DeepSeek-V3 tăng điểm từ 4% lên 37% ở Level 2.

Kỹ thuật 2: Iterative Refinement with Feedback (Đã trình bày trong bài khảo sát trước)

Hệ thống không sinh code mà không có kiến thức từ các lần trước, mà nó nhồi ngược thông báo lỗi biên dịch (Execution - E) và dữ liệu thời gian chạy (Profiler - P) vào Prompt để LLM tự sửa lỗi qua nhiều vòng, ví dụ như lên tới 10 vòng. Điều này giúp các mô hình suy luận như DeepSeek-R1 tự sửa lỗi biên dịch tốt hơn, tỷ lệ code chạy nhanh từ 36% lên tới 72% ở Level 2.



*Hình 11. Giải pháp **Iterative Refinement with Feedback***

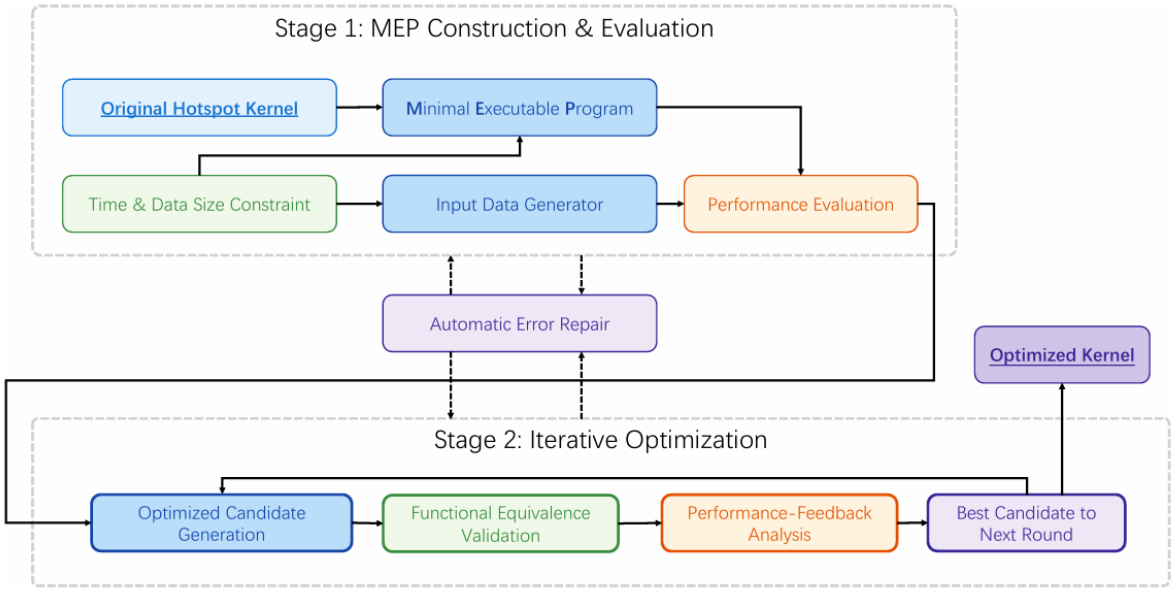
Kỹ thuật 3: Hardware-aware Prompting:
Bài báo thử cung cấp các ví dụ mẫu (Few-shot) về kỹ thuật tiling/fusion và thông số chi tiết của GPU (TFLOPS, băng thông bộ nhớ, kiến trúc) vào Prompt. LLM bắt đầu áp dụng Tiling cho 77% các bài toán nhân ma trận và tự quản lý bộ nhớ dùng chung (Shared Memory). Các mô hình như R1 thậm chí cố gắng gọi các tập lệnh phần cứng chuyên biệt (như ***wmma***).

c. *Những issue chưa giải quyết:*

Functional Correctness Bottleneck:
Mặc dù vòng lặp phản hồi giúp LLM sửa lỗi biên dịch tốt (đạt > 90% code chạy được sau 10 vòng), phần lớn các đoạn code này vẫn **sai kết quả toán học**. Nguyên nhân bài báo nêu là do phản hồi lỗi logic, ví dụ kết quả tensor không khớp không đủ chi tiết và, khiến LLM không biết phải sửa logic thuật toán ở dòng nào.

Tensor Core Utilization:
Các tập lệnh Tensor Core là quan trọng trong việc tối ưu code kernel. Mặc dù đã được cung cấp thông số phần cứng H100, các LLM hầu hết đều viết sai cú pháp hoặc gây lỗi biên dịch khi cố gắng sử dụng các lệnh này. Xuyên suốt các thử nghiệm, nhóm nghiên cứu ghi nhận không tìm thấy trường hợp nào sử dụng thành công các lệnh tensor core.

2. MEP Framework [3]



Hình 12.Pipeline chính của MEP Framework

a. *Ưu điểm:*

Giải quyết vấn đề phải build kernel lại từ đầu: Việc cô lập kernel và xây đựng thành một chương trình MEP nhỏ gọn giúp vòng lặp chạy thử và sửa lỗi diễn ra cực kỳ nhanh chóng, để tránh mỗi lần biên dịch lại toàn bộ để chạy thử có thể mất hàng giờ.

Performance Pattern Inheritance: Thay vì bắt đầu lại từ đầu ở mỗi vòng lặp, MEP tóm tắt các kỹ thuật đã thành công ở vòng trước (ví dụ: kích thước block tốt nhất, cách dùng shared memory) và đưa nó vào Prompt của vòng sau.

b. *Nhược điểm:*

Phụ thuộc vào khả năng code của LLM: Để tạo ra MEP, LLM phải tự viết thêm các đoạn code như khởi tạo bộ nhớ copy dữ liệu Host-to-Device. Nếu LLM viết sai, kernel có thể bị biên dịch lỗi, không thể tiến hành quy trình tối ưu phía sau.

Dùng 1 LLM: Dùng một luồng LLM duy nhất để phân tích lỗi, sinh code, lên chiến lược tối ưu khiến LLM dễ bị quá tải ngữ khi xử lý các kernel dài.

c. *Một số issue khác:*

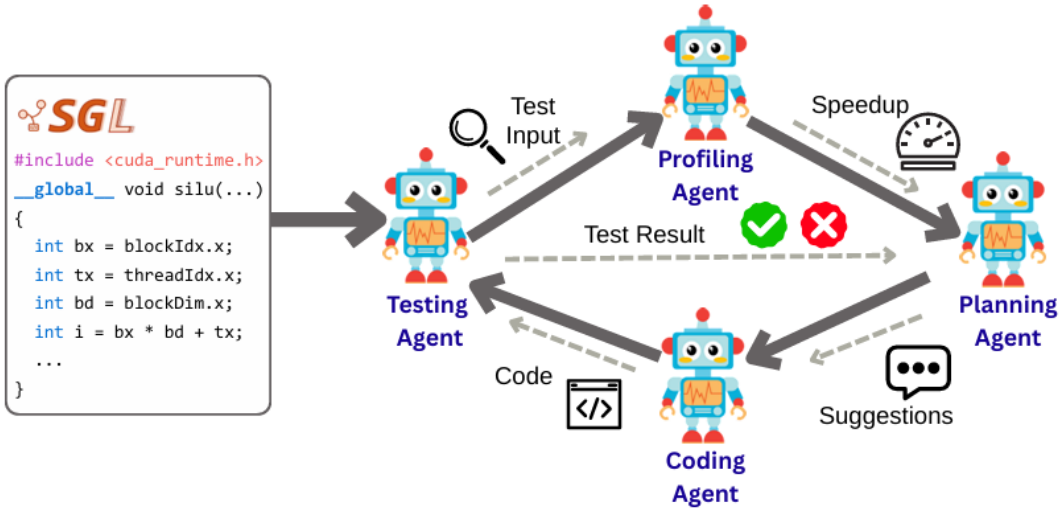
Issue 1: Vấn đề khi trích xuất với các mã nguồn có các ràng buộc khác phức tạp hơn

Việc trích xuất một đoạn kernel, nhưng để biến nó thành một MEP chạy được thì rất khó nếu kernel đó bị phụ thuộc bởi nhiều cấu trúc dữ như global structs, classes, hoặc các thư viện nội bộ phức tạp của ứng dụng gốc. LLM thường thiếu global context để tái tạo lại chính xác các ràng buộc này, khiến việc tạo MEP bị thất bại ngay từ bước đầu tiên.

Issue 2: Code thực tế thì có Performace thấp hơn

Khi chạy trên bộ test PolyBench gồm các thuật toán toán học thuần, framework MEP đạt mức tăng tốc là **5.05x**. Nhưng khi áp dụng vào các hotspot kernel của các ứng dụng thực tế như LAMMPS, tốc độ tăng tốc trung bình chỉ còn **1.25x**. Ta có thể rút ra rằng framework MEP này giải quyết rất tốt các bài tập thuật toán đơn thuần, hoặc có sẵn, nhưng lại chưa tối ưu trong việc phân tích lại cấu trúc của các đoạn code production phức tạp do con người viết.

3. ASTRA Framework [4]



Hình 15. Overview về Astra Framework

a. *Ưu điểm:*

Chiến lược dung 4 Agents giải quyết sự quá tải context: Việc chia quá trình tối ưu thành 4 tác nhân chuyên biệt (Testing, Profiling, Planning, Coding) giúp mỗi LLM chỉ phải tập trung vào một nhiệm vụ duy nhất.

Điều này tránh được hiện tượng LLM bị hallucination hoặc nhầm lẫn giữa logic code và con số hiệu năng như trong mô hình Single-agent.

Tính ứng dụng thực tiễn cực cao: nó lấy các đoạn code CUDA đang chạy trong hệ thống thực tế như SGLang để tối ưu hiệu năng.

Hiệu quả tốt: Astra hoạt động hiệu quả (đạt mức tăng tốc trung bình 1.32x) chỉ bằng kỹ thuật Zero-shot prompting với mô hình GPT-4o-mini mà không cần tốn thêm chi phí để Fine-tuning hay huấn luyện dùng Reinforcement Learning.

b. *Nhược điểm:*

Chi phí và độ trễ cao của hệ thống Multi-Agent: Việc duy trì một vòng lặp gồm 4 Agents gọi nhau trong N vòng lặp đòi hỏi lượng API call rất lớn. Thời gian chờ LLM phản hồi qua lại sẽ lâu hơn và tốn kém tài nguyên tính toán hơn rất nhiều so với phương pháp Single-agent.

c. *Các issue khác:*

Issue 1: Vấn đề phụ thuộc vào con người ở khâu tiền xử lý và hậu xử lý:

Để Astra có thể tối ưu, con người phải tiền xử lý bằng cách trích xuất các kernel từ framework SGLang thành các file riêng. Sau khi tối ưu xong, con người lại phải tự tay bổ sung thêm một số thứ để đưa kernel đó lại vào hệ thống. Bài báo nói rằng việc tự động hóa khâu này là không dễ dàng do sự phức tạp của các dependency nội bộ trong các framework hiện đại. Điều này giới hạn khả năng mở rộng của hệ thống lên nhiều kernel hơn.

Issue 2: "*Testing Agent Trust Issue*"

Astra có Testing Agent tạo ra các test case trong quá trình tối ưu, nhưng ở bước đánh giá cuối cùng, các tác giả không dám dùng test case của Agent này: *"the final evaluation relies on manually designed test cases to ensure high confidence in functional validation"* (Đánh giá cuối cùng vẫn phải dựa vào test case do con người thiết kế thủ công để đảm bảo độ tin cậy). LLM vẫn chưa đủ khả năng tự động sinh ra các bộ test cases cho tất cả các trường hợp.

4. SwizzlePerf framework [1]

a. *Ưu điểm:*

Hardware-Awareness: SwizzlePerf trích xuất các thông số phần cứng như số lượng XCD, CU, L2 cache và đưa vào Prompt. Điều này giúp LLM có cơ sở toán học để viết các công thức tối ưu không gian chính xác.

Tập trung rối ưu hướng mục tiêu phần cứng thay vì dựa vào Runtime: Bài báo SwizzlePerf framework cho rằng thời gian chạy runtime là một thước đo có nhiều noise. Còn SwizzlePerf thì tập trung làm cho LLM tối ưu một chỉ số phần cứng cụ thể (ở đây là Tỷ lệ trúng L2 Cache - L2 hit rate, giúp LLM dễ dàng đo lường sự tiến bộ qua từng vòng lặp.

b. *Nhược điểm:*

Phạm vi hẹp: Framework này hiện tại chỉ tập trung vào một kỹ thuật là "Swizzling" (thay đổi thứ tự ánh xạ của các block tính toán vào các vi xử lý) để tối ưu bộ nhớ. Nó không can thiệp vào các logic tính toán (như dùng fast math hay gộp vòng lặp) như framework Astra. Ngoài ra, việc tối ưu kernel còn nằm ở nhiều khía cạnh khác chứ không phải chỉ Cache - L2 hit rate.

Phụ thuộc phần cứng cụ thể: Framework được thiết kế và thử nghiệm với công cụ ***rocprof*** và kiến trúc ***chiplet của AMD***. Nên nếu mang framework này sang kiến trúc NVIDIA sẽ cần phải viết lại khâu trích xuất ngữ cảnh phần cứng.

c. *Các issue khác:*

Issue 1: Hardware-Overload và Overfitting:

SwizzlePerf cung cấp thông tin phần cứng cho LLM, nhưng khi thử cho toàn bộ tài liệu kiến trúc dài 10,000 token vào Prompt, performace giảm. LLM bị nhiễu và sinh ra các đoạn code chứa các phép dịch bit phức tạ mà bị overfit, tức là nó chạy cực nhanh ở một kích thước ma trận cụ thể, nhưng lập tức bị lỗi sai kết quả nếu đổi kích thước đầu vào.

Issue 2: Vấn đề trong xác định Bottleneck Metric:
Bài báo đặt ra metric mặc định là mục tiêu tối ưu cho LLM là tỷ lệ trúng L2 Cache. Tuy nhiên, trong môi trường thực tế, khi gặp một kernel hoàn toàn lạ, làm sao hệ thống biết được nó đang bị nghẽn ở L2 Cache, hay đang bị Register Pressure, hay do Memory Bandwidth, không phải lúc nào cũng là do L2 Cache. Bài báo chưa đưa ra được cơ chế để LLM tự động chẩn đoán xem nên chọn chỉ số nào làm mục tiêu tối ưu trước khi bắt tay vào sinh code vì dù sao bài báo cũng đang tập trung vào tỷ lệ trúng L2 Cache.

5. KernelBand framework [2]

a. *Ưu điểm:*

Điều hướng LLM theo cơ sở toán học và Machine Learing: Thay vì để LLM tự do sinh mã mà sẽ dễ sinh ra mã lỗi hoặc chậm, KernelBand điều hướng LLM theo một policy là thuật toán Multi-Armed Bandit - MAB đã được chứng minh là có regret bound tối ưu.

Tiết kiệm tài nguyên đo lường: Mặc dù phải dùng Nsight Compute, hệ thống tối ưu hóa chi phí bằng cách chỉ đo lường trên các kernel đại diện (ở đây là trung tâm cụm centroid) của từng cụm thay vì đo toàn bộ.

b. *Nhược điểm:*

Bị giới hạn bởi Tập các chiến lược định sẵn: KernelBand bắt buộc phải định nghĩa trước một tập hữu hạn các chiến lược $\boldsymbol{S}$. Vì vậy khả năng sáng tạo ra những chiến lược tối ưu hoàn toàn mới của LLM sẽ bị hạn chế và gần như không có.

Cần tinh chỉnh siêu tham số (Hyperparameter tuning): Hệ thống có các tham số như số lượng cụm K, chu kỳ phân cụm τ, và hệ số khám phá c và chọn sai các tham số này có thể làm giảm hiệu năng.

c. *Một số issue khác:*

Issue: Rủi ro phân mảnh dữ liệu khi phân cụm
Việc phân cụm sinh ra một vấn đề là chọn bao nhiêu cụm $\boldsymbol{K}$ là hợp lý. Nếu chia thành quá nhiều cụm , dữ liệu lịch sử sẽ bị phân mảnh. Thuật toán MAB sẽ không có đủ dữ liệu trên mỗi cụm để học được đâu là

chiến lược tốt, dẫn đến việc ra quyết định bị sai lệch và làm giảm tốc độ hội tụ. Bài báo có đề xuất ***một cách giải quyết*** đơn giản là **Khuyến nghị cố định *K=3*** như một điểm cân bằng tối ưu cho, đồng thời chỉ ra rằng nếu dữ liệu chiến lược ít, thì việc dồn vào 1-2 cụm lại tốt hơn.

V. So sánh giữa nhóm MEP – Astra và nhóm SwizzlePerf – KernelBand

1. Nhóm Astra và MEP

MEP: Chỉ quan tâm đến thời gian chạy cuối cùng, sử dụng Trimmed Mean để loại nhiễu và lưu lại các mẫu code chạy nhanh nhưng nó không học được tại sao mẫu code đó lại chạy nhanh trên kiến trúc đó.

Astra: Mặc dù có riêng một Profiling Agent, nhưng Agent này cũng chỉ đo lường thời gian thực thi trên các kích thước tensor thực tế. Planning Agent dựa vào thời gian này để đưa ra lời khuyên, nhưng nó không được cung cấp thông số cụ thể của GPU. LLM phải tự dựa vào trí nhớ từ lúc được huấn luyện để đoán xem phần cứng hoạt động ra sao.

2. Nhóm KernelBand và SwizzlePerf

SwizzlePerf: Chủ động trích xuất cấu hình vật lý của card đồ họa (Số lượng vi xử lý XCD, số lượng Compute Units, dung lượng bộ nhớ đệm L2). Nó cung cấp trực tiếp các con số này vào Prompt để LLM tính toán công thức toán học chia block hay Swizzling sao cho vừa khít với dung lượng Cache. Nó nhắm vào chỉ số Tỷ lệ trúng L2 Cache chứ không xét đến thời gian chạy tổng thể.

KernelBand: Tổng quát và hiệu quả hơn bằng cách dùng công cụ đo lường cấp thấp là Nsight Compute để quét các chỉ số phần cứng (số thanh ghi/luồng, shared memory, tỷ lệ bão hòa băng thông DRAM/L2/SM). Nó dùng các thông số này để cấm LLM áp dụng các chiến lược vô ích, đồng thời thuật toán được thiết kế để khám phá các chiến lược mới mà không dùng đi dùng lại một chiến lược quá nhiều lần.

TÀI LIỆU THAM KHẢO

[1] SwizzlePerf: Hardware-Aware LLMs for GPU Kernel Performance Optimization.

Truy cập: https://arxiv.org/abs/2508.20258

[2] KernelBand: Steering LLM-based Kernel Optimization via Hardware-Aware Multi-Armed Bandits.

Truy cập: https://arxiv.org/abs/2511.18868

[3] GPU Kernel Optimization Beyond Full Builds: An LLM Framework with Minimal Executable Programs.

Truy cập: https://arxiv.org/abs/2512.22147

[4] Astra: A Multi-Agent System for GPU Kernel Performance Optimization

Truy cập: https://arxiv.org/abs/2509.07506

[5] KernelBench: Can LLMs Write Efficient GPU Kernels?

Truy cập: https://arxiv.org/abs/2502.10517